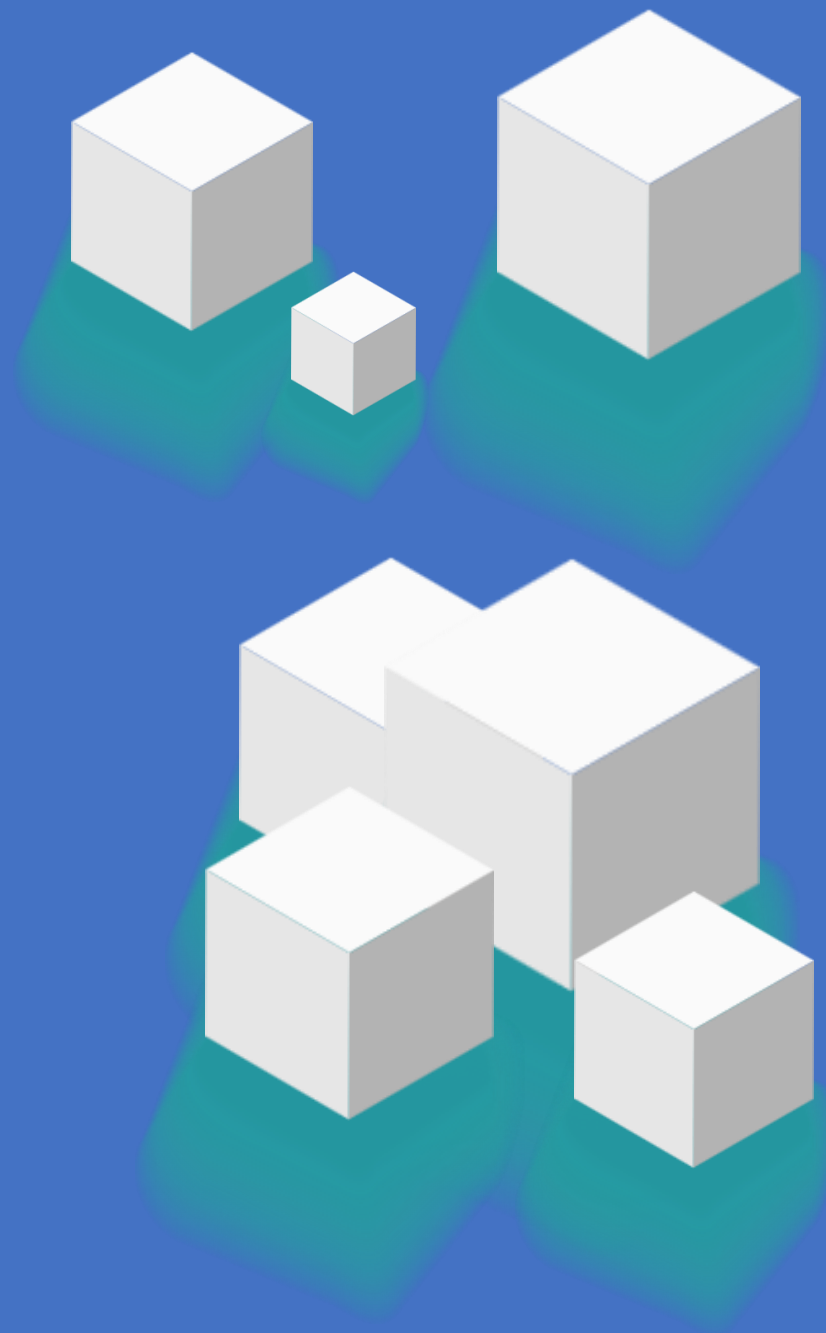
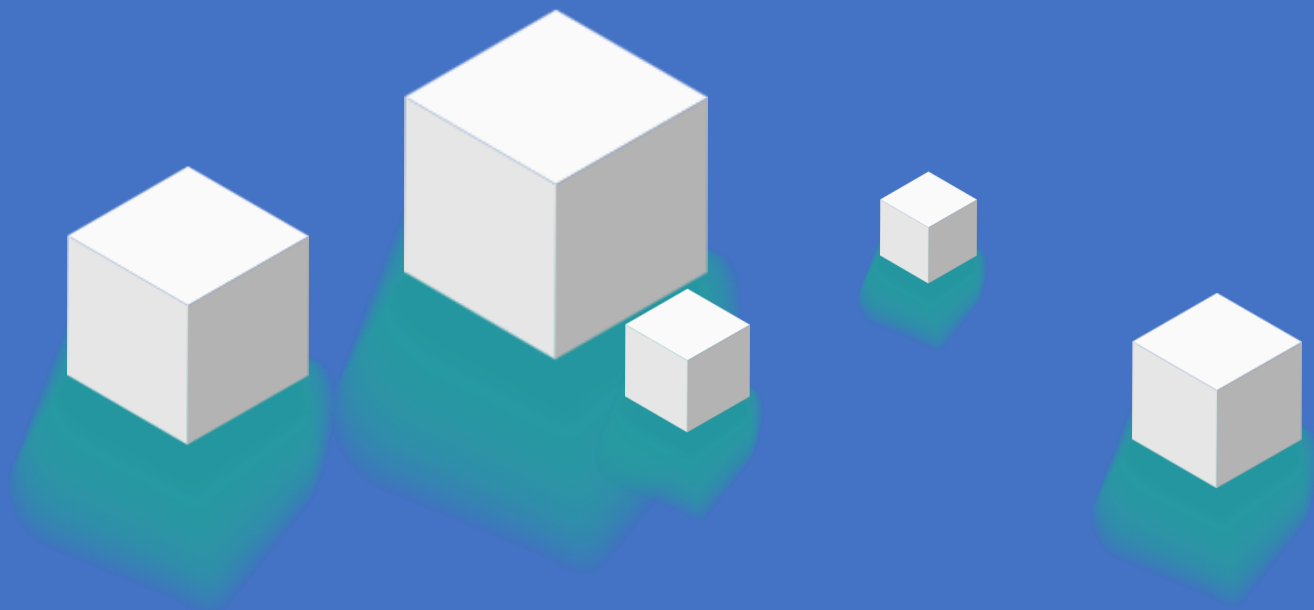


AI框架设计与选型



>> 今天的学习目标

AI框架设计与选型

- AI Agent的核心问题

LLM统一接口

工具注册与调度

Context管理机制

中枢的编排

- 主流Agent框架

LangChain 全能型LLM应用框架

LlamaIndex - 数据驱动的 RAG 专家

Qwen-Agent - 轻量级的全能选手

AutoGen - 多智能体框架

CASE: 多文件智能问答Agent

AI Agent的核心问题

Thinking: 都有哪些主流的AI Agent框架?

LangChain: 全能型框架, 管道式编排与丰富生态

Qwen-Agent: 轻量级全能选手, 工具调用与代码解释器

LlamaIndex: 数据驱动的 RAG 专家

AutoGen: 多智能体协作的

AI Agent的核心问题

Thinking: 编写一个AI Agent框架, 需要解决哪些核心问题?

1. LLM适配层 (大脑)
2. 工具注册与调度 (双手)
3. Context管理 (记忆)
4. 控制流编排 (中枢)

1. 大脑的适配层： LLM 统一接口与 Prompt 管理

1. 大脑的适配层： LLM 统一接口与 Prompt 管理

Model Adapter (适配器模式)

=> 抹平不同模型 (OpenAI, DeepSeek, Qwen) 的 API 差异, 各框架的实现方式:

```
# ===== LangChain (v0.3) =====  
from langchain_community.chat_models  
import ChatTongyi  
  
llm = ChatTongyi(  
    model_name="deepseek-v3",  
    dashscope_api_key=api_key  
)
```

```
# ===== Qwen-Agent =====  
llm_cfg = {  
    'model': 'deepseek-v3',  
    'model_server':  
        'https://dashscope.aliyuncs.com/compatibl  
e-mode/v1',  
    'api_key':  
        os.getenv('DASHSCOPE_API_KEY'),  
    'generate_cfg': {'top_p': 0.8}  
}
```

```
# ===== LlamaIndex =====  
from llama_index.llms.dashscope import  
DashScope  
  
llm = DashScope(  
    model="deepseek-v3",  
    api_key=api_key,  
    temperature=0.7,  
)
```

1. 大脑的适配层：LLM 统一接口与 Prompt 管理

框架	LLM 封装方式	特点
LangChain	ChatTongyi / ChatOpenAI 类	丰富的模型适配器，统一接口
Qwen-Agent	llm_cfg 字典配置	配置式，支持多种 model_server
LlamaIndex	DashScope / OpenAI 类	与 Settings 全局配置结合

框架层做了一个中间层，把统一的指令（如 `invoke("你好")`）翻译成特定模型的 API 调用。

LLM 统一接口的作用：

- 统一调用方式
- 统一了参数配置（如 temperature）
- 输出格式（统一转为 Message 对象）

1. 大脑的适配层： LLM 统一接口与 Prompt 管理

Prompt Engineering 工程化

System Message 的动态注入，将人设与上下文解耦：

```
# LangChain 的 PromptTemplate
from langchain_core.prompts import
ChatPromptTemplate, MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{input}")
])
```

```
# Qwen-Agent 的 system_message
system_instruction = '''你是一个乐于助人的AI助手。
在收到用户的请求后，你应该：
- 首先绘制一幅图像，得到图像的url，
- 然后运行代码下载该图像。
你总是用中文回复用户。'''
```

人设 (角色定义) 与任务流程 (上下文指令) 分离，便于复用和维护。

2. 双手的标准化：工具注册与调度 (Tool Registry)

Thinking: LLM 只能输出文本，怎么让它去执行 Python 函数？

各框架提供了不同的工具注册方式：

模式	框架	特点
@tool 装饰器	LangChain	最简洁，docstring 自动解析
@register_tool + 类	Qwen-Agent	显式参数定义，结构清晰
FunctionTool 封装	LlamaIndex	强类型约束，适合复杂工具

2. 双手的标准化：工具注册与调度 (Tool Registry)

三大框架工具注册对比

```
# ===== LangChain @tool 装饰器 =====  
from langchain_core.tools import tool  
@tool  
def ping_tool(target: str) -> str:  
    """检查本机到指定主机名或IP地址的网络连通性。  
    参数:  
        target: 目标主机名或IP地址  
    返回:  
        模拟的ping结果  
    """  
    if "unreachable" in target:  
        return f"Ping {target} 失败"  
    return f"Ping {target} 成功"
```

LangChain @tool 的优势：

自动从 docstring 解析工具描述和参数说明

支持类型注解，LLM 自动理解参数类型

一行装饰器，零配置即可使用

2. 双手的标准化：工具注册与调度 (Tool Registry)

```
# ===== Qwen-Agent @register_tool =====  
@register_tool('my_image_gen')  
class MyImageGen(BaseTool):  
    description = 'AI 绘画服务，输入文本描述，返回图像 URL'  
    parameters = [{  
        'name': 'prompt',  
        'type': 'string',  
        'description': '期望的图像内容的详细描述',  
        'required': True  
    }]  
  
    def call(self, params: str, kwargs) -> str:  
        prompt = json5.loads(params)['prompt']  
        return json5.dumps({'image_url': f'https://...'})
```

```
# ===== LlamaIndex FunctionTool =====  
def retrieve_documents(query: str) -> str:  
    """从文档中检索相关信息"""  
    response = query_engine.query(query)  
    return str(response)  
  
retrieve_tool =  
FunctionTool.from_defaults(fn=retrieve_documents)
```

Summary (工具注册与调度)

Thinking: LLM 是如何看见工具的?

框架会将 Python 函数的 name、docstring (功能描述) 和 type hints (参数类型) 转换成 JSON Schema 喂给 LLM。

LangChain: 也就是 @tool, 主要是在使用 Python 原生特性, 最符合直觉。

Qwen-Agent: @register_tool, 使用显式定义, 强约束, 适合复杂参数。

3. 记忆的存储：Context 管理机制

Thinking: 记忆系统的架构是怎样的？

1) 短期记忆 (Window)

- 对话历史截断
- 滑动窗口策略
- 避免 Token 爆炸

2) 长期记忆 (RAG)

- VectorStoreIndex 向量索引
- 文档分块与 Embedding
- 相似度检索

3. 记忆的存储：Context 管理机制

三大框架记忆管理对比

```
# LangChain 短期记忆 (RunnableWithMessageHistory)
from langchain_core.chat_history import
InMemoryChatMessageHistory
from langchain_core.runnables.history import
RunnableWithMessageHistory

# 会话存储
store = {}

def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]
```

```
# 创建带记忆的对话链
conversation = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="input",
    history_messages_key="history"
)

# 使用时指定 session_id
config = {"configurable": {"session_id": "user_123"}}
output = conversation.invoke({"input": "Hi!"}, config=config)

session_id 机制支持多用户并发会话
MessagesPlaceholder 自动注入历史到 Prompt
```

3. 记忆的存储： Context 管理机制

```
# === Qwen-Agent 短期记忆 (messages 列表) ===
messages = [] # 对话历史
messages.append({'role': 'user', 'content': query})
for response in bot.run(messages=messages):
    pass
messages.extend(response) # 追加响应
```

```
# ==== LlamaIndex 长期记忆 (VectorStoreIndex) ====
index = VectorStoreIndex.from_documents(documents)
# 持久化
index.storage_context.persist(persist_dir="./storage")
```

框架	短期记忆	长期记忆
LangChain	RunnableWithMessageHistory + session_id	需集成 VectorStore
Qwen-Agent	messages 列表手动管理	files 参数加载文档
LlamaIndex	Agent chat() 内置	专业级 VectorStoreIndex

Summary (记忆的存储)

Thinking: 为什么需要进行Context管理?

LLM 是无状态的, 它记不住你说过什么, 且 Context Window (上下文窗口) 是昂贵的资源。

=> 有限注意力的管理。

短期记忆:

- Session ID 很重要, 它是多用户并发的基础。
- 滑动窗口策略——只保留最近 N 轮, 防止 Token 爆炸。

长期记忆:

- 这是 RAG 的范畴, 利用向量数据库进行相关性检索, 而非时间顺序回忆。

4. 中枢的编排：控制流设计 (Orchestration)

Thinking：我们为什么需要控制流的编排？

复杂的任务不能靠 LLM 一口气说完，需要拆解步骤。

模式	说明	适用场景
管道模式 (Pipeline)	LangChain 的 prompt llm parser 链式调用	线性处理流程
单人模式 (Loop)	经典 ReAct 循环：思考 -> 行动 -> 观察	单 Agent 完成任务
多人模式 (DAG)	接力赛：明确的执行顺序	流程化任务 (如投资决策)
多人模式 (Chat)	圆桌会议：自由讨论	开放式协作

4. 中枢的编排：控制流设计 (Orchestration)

LangChain 管道式编排 (LCEL)

===== 管道语法: prompt | llm =====

```
from langchain_core.prompts import PromptTemplate
```

```
prompt = PromptTemplate(
```

```
    input_variables=["product"],
```

```
    template="What is a good name for a company that  
makes {product}?",
```

```
)
```

使用管道符组合

```
chain = prompt | llm
```

invoke 调用

```
result = chain.invoke({"product": "colorful socks"})
```

LangChain LCEL (LangChain Expression Language) 特点:

- | 管道符: 直观的链式调用, 类似 Unix 管道
- invoke(): 统一的调用接口
- 支持流式输出、批处理、异步调用

LangChain 管道模式

Prompt => LLM => Parser

4. 中枢的编排：控制流设计 (Orchestration)

```
# ===== Agent 模式: create_agent =====  
from langchain.agents import create_agent  
  
# 定义工具  
tools = [ping_tool, dns_tool, calculator]  
  
# 创建 Agent (LangChain 1.x 新写法)  
agent = create_agent(llm, tools)  
  
# 使用 messages 格式调用  
result = agent.invoke({"messages": [{"user", "检查  
www.example.com 的连通性"}]})  
print(result["messages"][-1].content)
```

ReAct 循环 (Agent模式)

Thought思考

=> Action行动

=> Observation 观察

=> Thought思考

...

Summary (控制流的编排)

控制流的编排：

- Chain (链式)：由于输入确定，输出确定，像工厂流水线 (Pipeline) 。
- Loop (循环)：即 ReAct 模式 (思考-行动-观察-思考)， 像一个不断试错的实验员，直到任务完成。
- DAG (有向无环图)：像多人接力赛，有明确的前后依赖关系。

LangChain 全能型LLM应用框架

LangChain 定位

Thinking: LangChain的定位是怎样的?

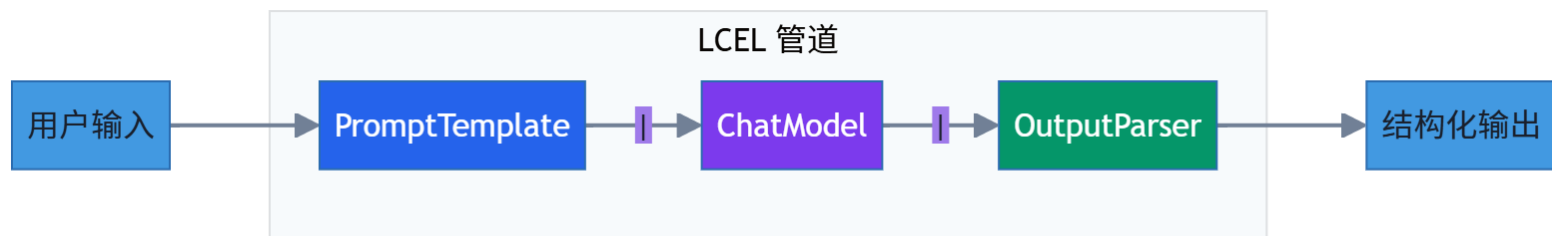
全能型LLM应用框架, 丰富的生态和组件, 适合各种复杂场景。

GitHub: github.com/langchain-ai/langchain

1. 核心特性： LCEL 管道语法

核心特性： LCEL 管道语法

LangChain Expression Language (LCEL) 是 LangChain 1.x 的核心创新，使用 | 管道符连接组件



LangChain 采用组件化的方式，核心优势是把 Prompt、Model、Memory、Retriever 都做成了标准积木（Runnable）。

1. 核心特性： LCEL 管道语法

基础管道示例

```
from langchain_core.prompts import PromptTemplate
from langchain_community.llms import Tongyi
```

```
# 加载模型
```

```
llm = Tongyi(model_name="qwen-turbo",
dashscope_api_key=api_key)
```

```
# 创建 Prompt Template
```

```
prompt = PromptTemplate(
    input_variables=["product"],
    template="What is a good name for a company that
makes {product}?",
)
```

```
# 管道语法组合
```

```
chain = prompt | llm
```

```
# invoke 调用
```

```
result = chain.invoke({"product": "colorful socks"})
print(result)
```

2. 工具注册: @tool 装饰器

LangChain 的 @tool 装饰器是最简洁的工具注册方式, 自动从 docstring 解析工具描述:

```
from langchain_core.tools import tool
```

```
@tool
```

```
def ping_tool(target: str) -> str:
```

```
    """检查本机到指定主机名或IP地址的网络连通性。
```

```
    参数:
```

```
        target: 目标主机名或IP地址
```

```
    返回:
```

```
        模拟的ping结果
```

```
    """
```

```
    if "unreachable" in target:
```

```
        return f"Ping {target} 失败: 请求超时。"
```

```
    return f"Ping {target} 成功: 延迟 20ms。"
```

```
@tool
```

```
def dns_tool(hostname: str) -> str:
```

```
    """解析给定的主机名, 获取其对应的IP地址。
```

```
    参数:
```

```
        hostname: 要解析的主机名
```

```
    返回:
```

```
        模拟的DNS解析结果
```

```
    """
```

```
    if hostname == "www.example.com":
```

```
        return f"DNS 解析 {hostname} 成功: IP 地址是
```

```
93.184.216.34"
```

```
    return f"DNS 解析 {hostname} 失败: 找不到主机。"
```


3. 创建 Agent

LangChain 1.x Agent 创建方式

```
from langchain.agents import create_agent
from langchain_community.chat_models import ChatTongyi
```

加载模型 (使用 ChatModel 以支持 tool calling)

```
llm = ChatTongyi(model_name="deepseek-v3",
dashscope_api_key=api_key)
```

定义工具列表

```
tools = [ping_tool, dns_tool, calculator]
```

创建 Agent (LangChain 1.x 新写法)

```
agent = create_agent(llm, tools)
```

使用 messages 格式调用

```
result = agent.invoke({
    "messages": [("user", "我无法访问 www.example.com,
帮我诊断一下")]
})
```

获取最终回复

```
print(result["messages"][-1].content)
```

4. 带记忆的对话链

```
from langchain_core.prompts import ChatPromptTemplate,
MessagesPlaceholder

from langchain_core.chat_history import
InMemoryChatMessageHistory

from langchain_core.runnables.history import
RunnableWithMessageHistory

# 创建带历史记录的 prompt
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    MessagesPlaceholder(variable_name="history"),
    ("human", "{input}")
])

# 创建 chain
chain = prompt | llm

store = {} # 会话存储
```

```
def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

# 创建带记忆的对话链
conversation = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="input",
    history_messages_key="history"
)

# 使用时指定 session_id
config = {"configurable": {"session_id": "user_123"}}
output = conversation.invoke({"input": "Hi there!"}, config=config)
print(output.content)
```

Summary

LangChain 的核心优势：

- 生态丰富：支持 100+ 模型、50+ 向量数据库、大量预置工具
- LCEL 管道语法：直观的链式调用，支持流式/批处理/异步
- @tool 装饰器：最简洁的工具注册方式
- 完善的记忆管理：session_id 机制支持多用户并发

场景	说明
工具调用型 Agent	网络诊断、数据查询、API 调用等需要多工具协作的场景
多轮对话系统	客服机器人、智能助手等需要记忆上下文的场景
复杂流程编排	使用 LCEL 构建多步骤处理流程
快速原型开发	丰富的组件库，快速搭建 POC

LlamaIndex - 数据驱动的 RAG 专家

LlamaIndex 定位

Thinking: LlamaIndex的定位是怎样的？

为 LLM 装上私有数据的最强接口

如果不涉及复杂的多人协作，只是想基于文档问答，它是首选。

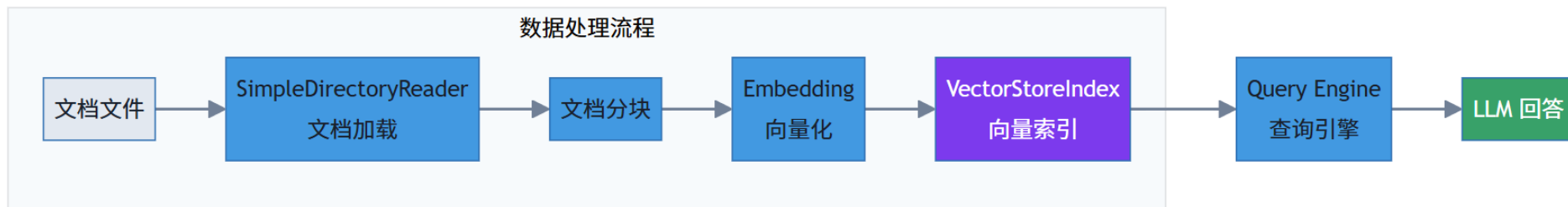
GitHub: github.com/run-llama/llama_index

1. 核心概念：Index优先

Index-First 哲学：

不同于 LangChain 关注流程，LlamaIndex 关注数据结构。

它认为 LLM 应用的核心瓶颈在于如何让 LLM 索引私有数据。



1. 核心概念: Index优先

加载文档并创建索引

```
from llama_index.core import VectorStoreIndex,
SimpleDirectoryReader

# 读取文档目录
reader = SimpleDirectoryReader('./docs')
documents = reader.load_data()
print(f"加载了 {len(documents)} 个文档")

# 创建向量索引
index = VectorStoreIndex.from_documents(documents)

# 持久化索引 (避免重复创建)
index.storage_context.persist(persist_dir="./storage")
```

从存储加载索引

```
from llama_index.core import StorageContext,
load_index_from_storage

# 检查索引是否已存在
persist_dir = "./storage"
if os.path.exists(persist_dir):
    # 从存储中加载索引 (快速启动)
    storage_context =
StorageContext.from_defaults(persist_dir=persist_dir)
    index = load_index_from_storage(storage_context)
    print("从存储加载索引成功")
else:
    # 创建新索引
    index = VectorStoreIndex.from_documents(documents)
```

2. ReAct Agent 与工具集成

将 retrieve_tool 作为一个函数插拔到 Agent 上：

```
from llama_index.core.agent import ReActAgent
from llama_index.core.tools import FunctionTool

# 创建查询引擎
query_engine = index.as_query_engine(similarity_top_k=5)

# 定义检索工具
def retrieve_documents(query: str) -> str:
    """从文档中检索相关信息"""
    response = query_engine.query(query)
    return str(response)
```

```
# 封装为 FunctionTool
retrieve_tool =
FunctionTool.from_defaults(fn=retrieve_documents)

# 创建 ReAct Agent
agent = ReActAgent.from_tools(
    tools=[retrieve_tool],
    llm=llm,
    verbose=True, # 显示思考过程
    system_prompt="你是一个乐于助人的AI助手，可以从文档中检索信息",
)
```


3. 完整流程示例

RAG 问答流程

```
# 配置 LLM 和 Embedding
```

```
llm = DashScope(model="deepseek-v3", api_key=api_key)
embed_model = DashScopeEmbedding(model_name="text-embedding-v2", api_key=api_key)
```

```
Settings.llm = llm
```

```
Settings.embed_model = embed_model
```

```
# 加载/创建索引
```

```
index = load_documents_and_create_index('./docs')
```

```
# 创建 Agent 和检索器
```

```
agent, retriever = create_agent(index, llm)
```

```
# 查询
```

```
query = "介绍下雇主责任险"
```

```
# 先展示召回的文档内容
```

```
retrieved_nodes = retriever.retrieve(query)
for i, node in enumerate(retrieved_nodes):
    print(f"文档片段 {i+1}: {node.text[:200]}...")
    print(f"相似度分数: {node.score}")
```

```
# 使用 Agent 回答
```

```
response = agent.chat(query)
print(response)
```

Summary

LlamaIndex 的核心优势:

- 一站式文档处理: 加载、分块、向量化、索引、检索
- 索引持久化: 避免重复创建, 快速启动
- 多种检索策略: 向量检索、关键词检索、混合检索
- 与 Agent 无缝集成: FunctionTool 封装查询引擎

场景	说明
企业知识库	内部文档、FAQ、操作手册的智能问答
合同审查助手	基于合同文档的条款检索与解读
学术论文分析	论文摘要、引用关系、知识图谱构建
客服机器人	产品手册、服务政策的实时检索回答

Qwen-Agent - 轻量级的全能选手

Qwen-Agent 定位

Thinking: Qwen-Agent的定位是怎样的?

阿里生态亲儿子, 对 Tool Use 和 Code Interpreter 支持好

轻量、灵活, 适合快速构建工具调用型 Agent。

GitHub: github.com/QwenLM/Qwen-Agent

1. 核心概念：

指令跟随优化：

Qwen-Agent 是专门为 Qwen 模型（及其强大的 Tool Calling 能力）定制的。

它不如 LangChain 抽象，但跑得快。

Code Interpreter (沙箱)：

这是它的杀手锏。不同于其他框架只调用外部 API，Qwen-Agent 内置了代码执行沙箱，允许 LLM 自己写 Python 代码来画图、做数据分析，自我修正错误。

Thinking：Qwen-Agent 适合什么场景？

想要快速搭建工具调用型或数据分析型 Agent，且不想处理复杂抽象层的开发者。

利用 Qwen-Agent 的长文本优势是强项。

2. 装饰器模式注册工具

自定义工具示例：图像生成

```
from qwen_agent.tools.base import BaseTool, register_tool

@register_tool('my_image_gen')
class MyImageGen(BaseTool):
    # description 告诉 LLM 这个工具的功能
    description = 'AI 绘画服务，输入文本描述，返回图像 URL'
    # parameters 定义输入参数
    parameters = [{
        'name': 'prompt',
        'type': 'string',
        'description': '期望的图像内容的详细描述',
        'required': True
    }]
```

```
def call(self, params: str, kwargs) -> str:
    # params 是 LLM 生成的 JSON 字符串
    prompt = json5.loads(params)['prompt']
    prompt = urllib.parse.quote(prompt)
    return json5.dumps({
        'image_url':
        f'https://image.pollinations.ai/prompt/{prompt}'
    }, ensure_ascii=False)
```

装饰器模式优势：

- @register_tool('name') 一行代码完成注册
- description 自动生成工具描述供 LLM 理解
- parameters 定义参数约束，LLM 自动解析

3. Code Interpreter: 沙箱代码执行

Qwen-Agent 内置 code_interpreter 工具，可以在沙箱环境中执行 Python 代码：

```
# 配置工具列表
tools = ['my_image_gen', 'code_interpreter']

# code_interpreter 是框架自带工具，可以：
# - 下载文件 (requests.get)
# - 处理图像 (PIL)
# - 数据分析 (pandas)
# - 绑图展示 (matplotlib)

system_instruction = """你是一个乐于助人的AI助手。
在收到用户的请求后，你应该：
- 首先绘制一幅图像，得到图像的url，
- 然后运行代码 requests.get 以下载该图像，
- 最后从给定的文档中选择一个图像操作进行图像处理。
用 plt.show() 展示图像。"""
```

能力	说明
代码生成	LLM 自动生成 Python 代码
沙箱执行	安全隔离环境运行代码
结果获取	捕获输出、图像、文件
错误修复	执行失败时自动修正重试

Code Interpreter 能力

4. 文件处理与多文档支持

加载多文件创建 Agent

```
from qwen_agent.agents import Assistant

# 获取文件夹下所有文件
file_dir = os.path.join('./', 'docs')
files = []
if os.path.exists(file_dir):
    for file in os.listdir(file_dir):
        file_path = os.path.join(file_dir, file)
        if os.path.isfile(file_path):
            files.append(file_path)

print('files=', files)
```

```
# 创建 Assistant, 传入文件列表
bot = Assistant(
    llm=llm_cfg,
    system_message=system_instruction,
    function_list=tools,
    files=files # 文档文件列表
)
```


5. 流式输出对话

流式输出对话

```
# 对话历史
messages = []

# 用户查询
query = "介绍下雇主责任险"
messages.append({'role': 'user', 'content': query})

# 流式输出
current_index = 0
for response in bot.run(messages=messages):
    # 获取增量内容
    current_response = response[0]['content'][current_index:]
    current_index = len(response[0]['content'])
    print(current_response, end="")
```

```
# 添加到对话历史, 支持多轮对话
messages.extend(response)
```

Summary (Qwen-Agent的适用场景)

场景	说明
数据分析	需要运行代码、绑图、生成报表
复杂工具调用链	多个工具协同完成任务
图像处理	生成、编辑、分析图像
文档问答	结合 Qwen 模型的长 Context 优势

Summary (Agent框架对比)

维度	LangChain	Qwen-Agent	LlamaIndex	AutoGen
核心定位	全能型框架	轻量工具调用	RAG 数据接口	多Agent协作
工具注册	@tool 装饰器	@register_tool 装饰器	FunctionTool 类	@register 装饰器
RAG 支持	需集成 VectorStore	基础文件读取	专业级向量索引	需自行集成
多Agent	LangGraph 支持	基础支持	需自行编排	原生GroupChat
代码执行	需集成	内置 code_interpreter	需集成	UserProxyAgent
记忆管理	RunnableWithMessageHistory	messages 列表	Agent chat()	GroupChat 自动
学习曲线	中等	简单	中等	中等
生态完整度	最丰富	阿里生态	RAG 社区	微软生态

Summary (Agent框架对比)

Thinking: 从业务场景的角度, 如何选择适合的Agent框架?

选 LangChain: 如果你要开发通用的 AI 应用, 需要灵活控制流程, 或者需要切换多种模型。

选 LlamaIndex: 如果你主要做 RAG (企业知识库), 手里有一堆 PDF/Word/Excel 要处理。

选 Qwen-Agent: 如果你主要用 Qwen 模型, 需要做数据分析 (Code Interpreter) 或处理超长文档 (1M Context) 。

选 AutoGen: 如果任务太复杂, 一个人 (Agent) 干不完, 需要团队 (多角色) 吵架/协作才能出结果。

CASE: 多文件智能问答Agent

CASE：多文件智能问答Agent

TO DO：搭建一个 保险产品智能问答Agent，用于帮助用户快速了解各类保险产品的详细信息。

加载了多个保险产品文档，包括：

- 雇主责任险
- 平安商业综合责任保险
- 企业团体综合意外险
- 财产一切险
- 施工保、装修保等

用户可以通过自然语言提问，系统会从文档中检索相关信息并给出回答。

CASE：多文件智能问答Agent

技术方案：RAG (检索增强生成)

RAG 的核心流程：

用户问题 → 向量检索 → 召回相关文档 → LLM 生成回答

Thinking：为什么需要 RAG？

- LLM 没有私有数据的知识
- 避免模型幻觉（编造信息）
- 回答可追溯到具体文档来源

CASE：多文件智能问答Agent（LangChain实现）

LangChain 实现

- LCEL 管道语法：使用 | 管道符连接组件
- 丰富的生态：支持多种文档加载器、向量数据库
- 灵活的组合：各组件可自由搭配

```
#### LLM 配置
```

```
from langchain_community.chat_models import ChatTongyi
```

```
llm = ChatTongyi(
```

```
    model_name="deepseek-v3",
```

```
    dashscope_api_key=DASHSCOPE_API_KEY
```

```
)
```


CASE：多文件智能问答Agent（LangChain实现）

文档加载与索引

```
from langchain_community.document_loaders import
DirectoryLoader, TextLoader

from langchain_community.vectorstores import FAISS

# 加载目录下所有 txt 文件
loader = DirectoryLoader(
    file_dir,
    glob="/*.txt",
    loader_cls=TextLoader,
    loader_kwargs={"encoding": "utf-8"}
)

documents = loader.load()

# 创建向量索引
vector_store = FAISS.from_documents(chunks, embeddings)
```

问答链 (LCEL 语法)

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

qa_prompt = ChatPromptTemplate.from_messages([
    ("system", "根据上下文回答问题...\n{context}"),
    ("human", "{question}")
])

# 管道语法
qa_chain = qa_prompt | llm | StrOutputParser()

# 调用
response = qa_chain.invoke({"context": context, "question":
    query})
```

CASE：多文件智能问答Agent（LangChain实现）

加载了 7 个文档
文本被分割成 15 个块
索引已保存到 ./langchain_storage

用户查询：介绍下雇主责任险

===== 召回的文档内容 =====

文档片段 1：
内容：【雇主责任险】
Q1 雇员意外事故给企业造成的损害有多大？甚至可能让一家公司倒闭！！
员工出外勤不幸遇到车祸被撞高位截瘫
公司赔偿近一百万
抽干公司流动资金
无力维持生产
走破产程序...

Q2 雇主责任险的保障范围和其他亮点
1. 保障
只要有用工需求的单位，都可以购买雇主责任险。
2. 保障范围：
死亡赔偿金
伤残赔偿金
医疗费用
误工费用（c款不赔）
法律诉讼费（c款不赔）
3. 其他亮点...
来源：docs\2-雇主责任险.txt

文档片段 2：
内容：Q4：雇主责任险的优势是什么（与团体意外伤害保险的区别）？
投保雇主责任险是符合国家法律法规要求的；意外险主要体现的是企业对雇员的福利，没有相关法律依据，如果雇主对雇员通过意外险给予了赔偿，雇员还是可以通过法律手段要求雇主继续给予赔偿的。

- ### **二、产品亮点**
- 1. ****风险转移****
 - 将企业依法承担的雇主责任（如《工伤保险条例》规定的赔偿）转嫁给保险公司，避免高额赔付导致经营危机。
 - ***案例***：员工意外身亡后，即使企业已购买团体意外险，家属仍可起诉企业索赔，雇主责任险能直接覆盖这部分法律赔偿责任。
 - 2. ****补充工伤保险不足****
 - 工伤保险不覆盖的部分（如停工留薪期工资、一次性伤残就业补助金等），雇主责任险可赔付。
 - 3. ****减少纠纷****
 - 快速理赔可降低员工与企业间的工伤争议，提升员工满意度。
 - 4. ****税务优惠****
 - 保费可税前列支，减少25%企业所得税。

### **三、与类似保险的区别**			
对比项	**雇主责任险**	**团体意外险**	**工伤保险**
法律依据	覆盖法定雇主责任	企业福利，无法律豁免	国家强制，但赔偿不全
保障内容	含误工费、职业病、诉讼费	仅意外身故/伤残/医疗	基础工伤待遇，企业仍需补差额
赔付对象	直接赔付给企业（雇主）	赔付给员工或家属	工伤保险基金赔付

- ### **四、适用场景**
- ****高风险行业****：建筑、制造、物流等工伤高发行业。
 - ****灵活用工****：覆盖临时工、学徒工等（即使未签劳动合同，存在事实劳动关系即可）。
 - ****补充保障****：即使已购工伤保险或团体意外险，仍可叠加投保以填补缺口。
- ### **五、注意事项**
- ****医疗费限制****：通常仅赔付医保范围内费用，需注意条款约定。
 - ****产品版本差异****：A/B/94款保障范围不同，需根据企业需求选择（如A款完全对标《工伤保险条例》）。

****总结****：雇主责任险是企业规避用工风险的重要工具，尤其适合希望合规降责、减少财务波动的雇主。如需具体方案，建议根据行业、工种和员工人数定制投保。

CASE：多文件智能问答Agent（LlamaIndex实现）

LlamaIndex 实现

- Index 优先：专注于数据索引和检索
- 一站式 RAG：文档加载、分块、索引、检索一体化
- ReAct Agent：内置思考-行动循环

LLM 和 Embedding 配置

```
from llama_index.llms.dashscope import DashScope
from llama_index.embeddings.dashscope import
DashScopeEmbedding
```

```
llm = DashScope(model="deepseek-v3", api_key=api_key)
embed_model = DashScopeEmbedding(model_name="text-
embedding-v2", api_key=api_key)
```

全局设置

```
Settings.llm = llm
Settings.embed_model = embed_model
```

CASE：多文件智能问答Agent（LlamaIndex实现）

文档加载与索引

```
from llama_index.core import VectorStoreIndex,  
SimpleDirectoryReader
```

一行代码加载整个目录

```
reader = SimpleDirectoryReader(file_dir)  
documents = reader.load_data()
```

创建向量索引

```
index = VectorStoreIndex.from_documents(documents)
```

持久化

```
index.storage_context.persist(persist_dir="./storage")
```

创建 Agent

```
from llama_index.core.agent import ReActAgent  
from llama_index.core.tools import FunctionTool
```

创建检索工具

```
def retrieve_documents(query: str) -> str:  
    response = query_engine.query(query)  
    return str(response)
```

```
retrieve_tool =
```

```
FunctionTool.from_defaults(fn=retrieve_documents)
```

CASE：多文件智能问答Agent（LlamaIndex实现）

```
# 创建 ReAct Agent

agent = ReActAgent.from_tools(

    tools=[retrieve_tool],

    llm=llm,

    verbose=True

)

# 对话

response = agent.chat(query)
```

===== 召回的文档内容 =====

文档片段 1:

内容：Q4：雇主责任险的优势是什么（与团体意外伤害保险的区别）？
投保雇主责任险是符合国家法律法规要求的；意外险主要体现的是企业对雇员的福利，没有相关法律依据，如果雇主对雇员通过意外险给予了赔偿，雇员还是可以通过法律手段要求雇主继续给予赔偿的。
雇主责任险可以承保职业性疾病；意外险无法承保。
雇主责任险可以承保误工费；意外险无法承保，仅能含有住院津贴。

Q5：雇主责任险的优势是什么（与工伤...）

元数据：{'file_path': 'D:\\推荐系统\\知乎\\新课程\\17-AI框架设计与选型\\CASE-RAG应用\\docs\\2-雇主责任险.txt', 'file_name': '2-雇主责任险.txt', 'file_type': 'text/plain', 'file_size': 4991, 'creation_date': '2026-01-27', 'last_modified_date': '2024-12-02'}
相似度分数：0.7230075122865547

文档片段 2:

内容：【雇主责任险】

Q1 雇员意外事故给企业造成的损害有多大？甚至可能让一家公司倒闭！！

员工出外勤不幸遇到车祸被撞高位截瘫

公司赔偿近一百万

抽干公司流动资金

无力维持生产

走破产程序...

Q2 雇主责任险的保障范围和其他亮点

1. 保障

只要有有用工需求的单位，都可以购买雇主责任险。

2. 保障范围：

死亡赔偿金

伤残赔偿金

医疗费用

误工费（c款不赔）

法律诉...

元数据：{'file_path': 'D:\\推荐系统\\知乎\\新课程\\17-AI框架设计与选型\\CASE-RAG应用\\docs\\2-雇主责任险.txt', 'file_name': '2-雇主责任险.txt', 'file_type': 'text/plain', 'file_size': 4991, 'creation_date': '2026-01-27', 'last_modified_date': '2024-12-02'}
相似度分数：0.7111800467440366

文档片段 3:

内容：可以，根据雇主责任险条款中的释义，雇员是指与被保险人签订有劳动合同或存在事实劳动合同关系，接受被保险人给付薪金、工资，年满十六周岁且不超过65周岁的人员及其他按国家规定审批的未满十六周岁的特殊人员，包括正式在册职工、短期工、临时工、季节工和徒工等。

llamaindex-agent-multi-files.py

CASE：多文件智能问答Agent（Qwen-Agent实现）

Qwen-Agent 实现

- 开箱即用：配置简单，快速上手
- 内置 WebUI：一行代码启动 Web 界面
- Code Interpreter：内置代码执行能力
- files 参数：直接传入文件列表

LLM 配置

```
llm_cfg = {  
    'model': 'deepseek-v3',  
    'model_server': 'https://dashscope.aliyuncs.com/compatible-mode/v1',  
    'api_key': os.getenv('DASHSCOPE_API_KEY'),  
    'generate_cfg': {'top_p': 0.8}  
}
```

CASE：多文件智能问答Agent（Qwen-Agent实现）

```
#### 自定义工具 (@register_tool)
from qwen_agent.tools.base import BaseTool, register_tool
@register_tool('my_image_gen')
class MyImageGen(BaseTool):
    description = 'AI 绘画服务'
    parameters = [{
        'name': 'prompt',
        'type': 'string',
        'description': '图像描述',
        'required': True
    }]
    def call(self, params: str, kwargs) -> str:
        prompt = json5.loads(params)['prompt']
        return json5.dumps({'image_url': f'https://...'})
```

```
#### 创建 Assistant
from qwen_agent.agents import Assistant

# 获取文档文件列表
files = get_doc_files()

# 创建 Assistant（files 参数直接传入文档）
bot = Assistant(
    llm=llm_cfg,
    system_message=system_instruction,
    function_list=['my_image_gen', 'code_interpreter'],
    files=files # 直接传入文件列表
)
```

CASE：多文件智能问答Agent（Qwen-Agent实现）

```
#### 启动 WebUI
```

```
from qwen_agent.gui import WebUI
```

```
WebUI(bot, chatbot_config=chatbot_config).run()
```

了解下雇主责任险



user

雇主责任险是一种专门为企业设计的保险产品，主要用于转移企业在员工因工作原因遭受意外伤害或职业病时的法律责任和经济赔偿风险。以下是雇主责任险的核心内容：

1.保障范围

- **死亡赔偿金**：员工因工死亡时，企业需支付的赔偿。
- **伤残赔偿金**：员工因工致残时，根据伤残等级支付的赔偿。
- **医疗费用**：员工因工受伤或患职业病产生的医疗费用。
- **误工费用**：员工因工受伤无法工作期间的工资补偿（部分产品不包含）。
- **法律诉讼费**：因工伤纠纷产生的法律费用（部分产品不包含）。

2.适用对象

- 所有用工单位，包括正式员工、临时工、季节工、学徒工等。

3.与其他保险的区别

- **与团体意外险的区别**：
 - 雇主责任险是企业对员工的法定责任，赔偿直接给企业；团体意外险是福利性质，赔偿给员工个人。
 - 雇主责任险涵盖职业病和误工费，团体意外险通常不包含。
- **与工伤保险的区别**：
 - 工伤保险是社会保险，雇主责任险是商业保险，可补充工伤保险未覆盖的部分（如误工费、诉讼费）。

Summary

维度	LangChain	LlamaIndex	Qwen-Agent
定位	全能型框架	RAG 专家	轻量级 Agent
文档加载	DirectoryLoader	SimpleDirectoryReader	files 参数
向量存储	FAISS/Chroma 等	VectorStoreIndex	内置
问答方式	LCEL 管道	ReActAgent	bot.run()
索引持久化	save_local()	persist()	无
Web UI	需集成 Gradio	需集成	内置 WebUI
代码执行	需集成	需集成	内置 code_interpreter

打卡：多文件智能问答Agent



基于你的业务场景，搭建多文件智能问答Agent，比如这里了针对 保险知识进行问答
可以采用任意Agent框架实现（选择其中之一即可）

- LangChain
- LlamaIndex
- Qwen-Agent

-  1-平安商业综合责任保险（亚马逊）
-  2-雇主责任险
-  3-平安企业团体综合意外险
-  4-雇主安心保
-  5-施工保
-  6-财产一切险
-  7-平安装修保
-  平安产险交通出行意外伤害保险（互联网版...
 平安产险交通工具意外伤害保险（互联网版...
 平安附加疾病身故保险条款
 平安境内紧急医疗救援服务条款
 平安企业团体综合意外险(互联网版)适用条款
 平安商业综合责任保险（亚马逊）

AutoGen 多智能体框架

AutoGen 多智能体框架

AutoGen 是微软开源的多智能体对话框架，用于构建多个 AI Agent 协作完成复杂任务。

它的核心理念是让 Agent 之间通过自然语言对话来协作，而非硬编码的函数调用。

<https://github.com/microsoft/autogen>

你也可以使用 Agent Framework

<https://github.com/microsoft/agent-framework>

2025 年 10 月起，微软将 AutoGen 置为维护模式——仅修漏洞，不再新增功能；所有新特性都做到 Agent Framework 上。

官方文档明确把 Agent Framework 称为下一代 Semantic Kernel 与 AutoGen，鼓励新项目直接迁移

Agent定义

Agent 是 AutoGen 的基本单元，每个 Agent 具备以下属性：

属性	说明
name	Agent 的名称标识，在对话中用于区分发言者
system_message	角色设定/提示词，定义 Agent 的职责和行为方式
llm_config	LLM 配置，包括模型、API Key、温度等参数
tools	可调用的外部工具函数，扩展 Agent 能力

Agent定义

投资委员会项目中的 Analyst Agent

```
from autogen import ConversableAgent

agent = ConversableAgent(
    name="AnalystAgent",          # 名称标识
    system_message="""你是投资委员会的分析师 ... """,
    llm_config={                  # LLM 配置
        "config_list": [{
            "model": "gemini-2.5-flash",
            "api_key": "your_api_key",
            "api_type": "google"
        }]
    },
    human_input_mode="NEVER",     # 不需要人工输入
)
```

Agent定义

常用 Agent 类型

类型	用途	特点
ConversableAgent	基础对话 Agent	最灵活，可完全自定义
AssistantAgent	助手 Agent	默认由 LLM 驱动，适合生成内容
UserProxyAgent	用户代理	可执行代码、调用工具、请求人工输入

项目中使用 ConversableAgent 创建了 4 个角色，区别在于是否注册工具函数

Agent定义

data_agent - 带工具的 Agent

```
agent = ConversableAgent(name="DataAgent", ...)
```

```
@agent.register_for_llm(description="获取股票的行情、估值、财务等综合数据")
```

```
@agent.register_for_execution()
```

```
def fetch_stock_data(query: str) -> str:
```

```
    return get_stock_data(query)
```

analyst_agent - 不带工具的 Agent

```
agent = ConversableAgent(name="AnalystAgent", ...)
```

无需注册工具，只靠 system_message 引导 LLM 输出分析结论

GroupChat (群聊)

GroupChat (群聊)

将多个 Agent 放在同一个对话中协作的容器

GroupChat

- |—— agents: [Agent1, Agent2, Agent3, Agent4]
- |—— max_round: 10 # 最大对话轮次
- |—— speaker_selection_method: # 发言者选择方式
 - |—— "round_robin" # 轮流发言 (固定顺序)
 - |—— "random" # 随机选择
 - |—— "auto" # LLM 自动选择下一个发言者

GroupChat (群聊)

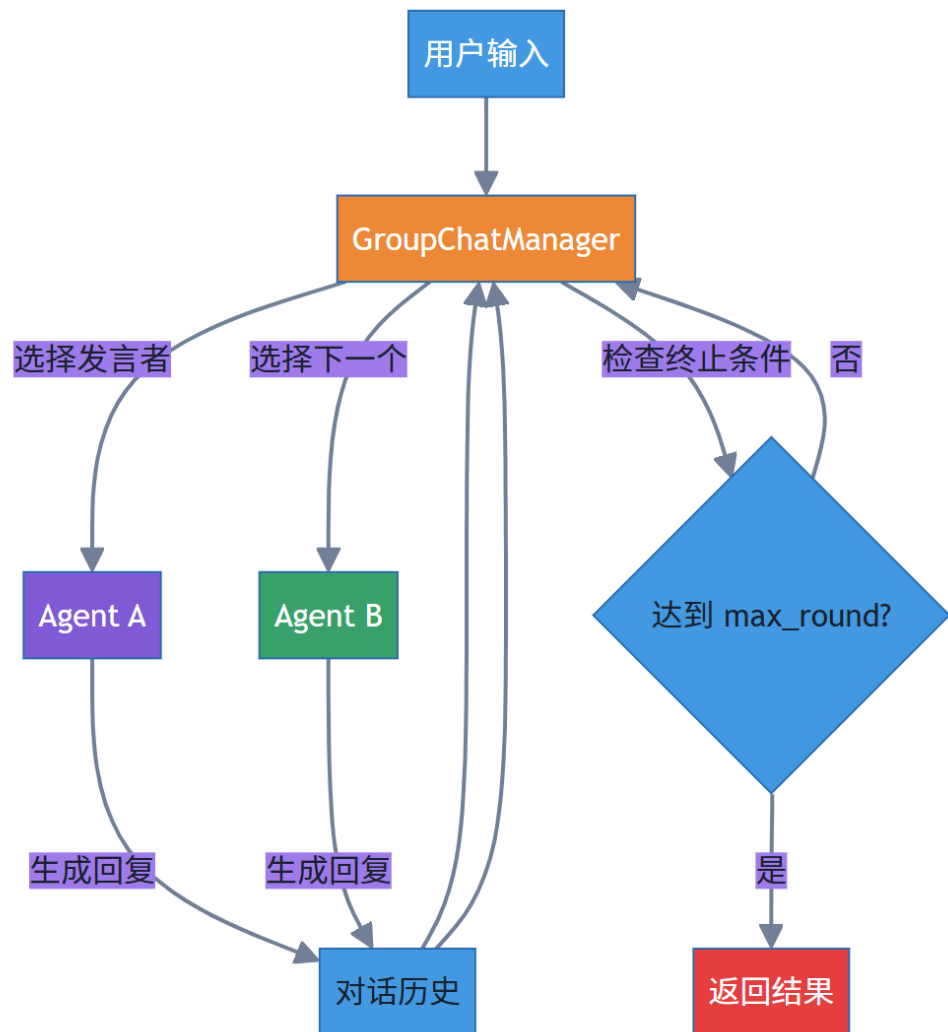
```
# __init__.py - InvestmentCommittee.analyze() 方法
from autogen import GroupChat, GroupChatManager

# 创建群聊, 设定发言顺序
group_chat = GroupChat(
    agents=[
        self.data_agent,    # 1. 数据员先获取数据
        self.analyst_agent, # 2. 分析师进行分析
        self.risk_agent,    # 3. 风控官评估风险
        self.trader_agent   # 4. 交易员给出建议
    ],
    messages=[],
    max_round=8,            # 4个Agent各发言1-2次
    speaker_selection_method="round_robin", # 按顺序轮流发言
)
```

```
# 创建群聊管理器
manager = GroupChatManager(
    groupchat=group_chat,
    llm_config=self.data_agent.llm_config,
)

# 发起对话
result = self.data_agent.initiate_chat(
    manager,
    message="用户查询: 分析一下宁德时代能不能买",
)
```

对话流程



对话流程：

- 1) 用户发起查询
- 2) GroupChatManager 根据策略选择第一个发言的 Agent
- 3) 被选中的 Agent 生成回复（可能调用工具）
- 4) 回复加入对话历史，所有 Agent 可见
- 5) GroupChatManager 选择下一个发言者
- 6) 重复 3-5，直到达到 max_round 或满足终止条件

发言者选择策略

发言者选择策略

策略	说明	适用场景
round_robin	按 agents 列表顺序轮流发言	流程明确的任务（如：数据→分析→风控→决策）
random	随机选择下一个发言者	头脑风暴、创意讨论
auto	由 LLM 判断谁最适合回答当前问题	开放式讨论、问答场景
自定义函数	完全控制选择逻辑	复杂业务流程、条件分支

如果你的任务有明确的执行顺序，用 round_robin；如果需要灵活讨论，用 auto。

投资委员会使用 round_robin 是因为投资决策有固定流程：

先获取数据 → 再分析 → 再风控 → 最后决策。

Summary

场景 1：企业知识库问答

推荐：LlamaIndex

- 专业的文档处理能力
- 多种检索策略（向量、关键词、混合）
- 索引持久化，支持增量更新

场景 2：快速 Demo / POC

推荐：Qwen-Agent

- 配置最简单
- 内置 WebUI，无需前端开发
- 开箱即用的工具（code_interpreter）

场景 3：复杂业务流程

推荐：LangChain

- LCEL 支持灵活的流程编排
- 丰富的组件生态
- 可与其他框架组合使用

场景 4：多人协作

推荐：AutoGen

- 多智能体协同对话
- 支持角色自定义与任务分工
- 内置群聊管理与执行控制



Thank You
Using data to solve problems